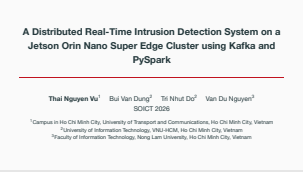
A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu[1] Bui Van Dung[2] Tri Nhut Do[2] Van Du Nguyen[3]

SOICT 2026

[1]Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam
[2]University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam
[3]Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam



Good morning everyone. We have two small boards. We split an attack detector across them. I will show what we gain, and what we lose. I have five results.

I. PROBLEM AND CONTRIBUTIONS

Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

What the literature does not tell us

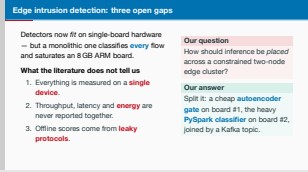
1. Everything is measured on a **single device**.
2. Throughput, latency and **energy** are never reported together.
3. Offline scores come from **leaky protocols**.

Our question

How should inference be *placed* across a constrained two-node edge cluster?

Our answer

Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2, joined by a Kafka topic.

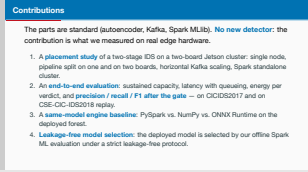


Past work has three gaps. One: it uses only one board. Two: it does not report speed and energy together. Three: test data often leaks into training. We fix all three on two real boards.

Contributions

The parts are standard (autoencoder, Kafka, Spark MLlib). **No new detector**: the contribution is what we measured on real edge hardware.

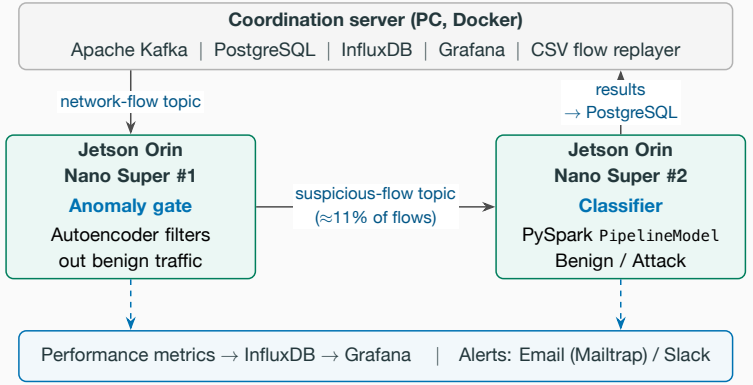
1. A **placement study** of a two-stage IDS on a two-board Jetson cluster: single node, pipeline split on one and on two boards, horizontal Kafka scaling, Spark standalone cluster.
2. An **end-to-end evaluation**: sustained capacity, latency with queueing, energy per verdict, and **precision / recall / F1 after the gate** — on CICIDS2017 and on CSE-CIC-IDS2018 replay.
3. A **same-model engine baseline**: PySpark vs. NumPy vs. ONNX Runtime on the deployed forest.
4. **Leakage-free model selection**: the deployed model is selected by our offline Spark ML evaluation under a strict leakage-free protocol.



We do not make a new model. All parts are standard. Our work is the measurement: where to put each part, how fast it gets, and what we lose.

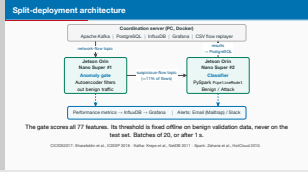
II. SYSTEM AND EVALUATION SETUP

Split-deployment architecture



The gate scores all 77 features. Its threshold is fixed *offline* on benign validation data, never on the test set. Batches of 20, or after 1 s.

CICIDS2017: Sharafaldin et al., ICISSP 2018 · Kafka: Kreps et al., NetDB 2011 · Spark: Zaharia et al., HotCloud 2010.



Here is our system. The PC runs Kafka, the database, and Grafana. Board one runs a small filter, we call it the gate. It passes about eleven percent of the traffic to board two. Board two runs the big classifier.

Deployment modes and what we measure

Single node full pipeline on one board, gate off or on.

A: Pipeline split gate on #1, classifier on #2 (our design).

A1: Split, one board gate and classifier as two processes on #2.

B: Horizontal both boards run the full role (gate off).

C: Spark cluster host as master, both boards as workers.

Every flow is followed from send to verdict, queueing included. **Sustained capacity**: the highest rate at which every flow gets its verdict within 10 s of the load.

Setup

2× Jetson Orin Nano Super
(6-core A78AE, 8 GB, ≈67 TOPS)
PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN
10,000 test-split flows, 15% attacks
+ 10,000 CSE-CIC-IDS2018 flows
Rate sweep 1–40 flows/s, 2 runs each
Energy via `tegrastats`

We test five setups. The key one is A1: two separate programs on one board. A uses two boards. We measure the time from sending a flow to getting the answer. All flows must finish within ten seconds.

III. RESULTS AND CONCLUSION

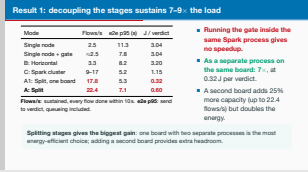
Result 1: decoupling the stages sustains 7–9× the load

Mode	Flows/s	e2e p95 (s)	J / verdict
Single node	2.5	11.3	3.04
Single node + gate	≈2.5	7.8	3.04
B: Horizontal	3.3	8.2	3.20
C: Spark cluster	9–17	5.2	1.15
A1: Split, one board	**17.8**	5.3	**0.32**
A: Split	**22.4**	**7.1**	**0.60**

Flows/s: sustained, every flow done within 10 s. **e2e p95**: send to verdict, queueing included.

- **Running the gate inside the same Spark process gives no speedup.**
- **As a separate process on the same board: 7×**, at 0.32 J per verdict.
- A second board adds 25% more capacity (up to 22.4 flows/s) but doubles the energy.

Splitting stages gives the biggest gain: one board with two separate processes is the most energy-efficient choice; adding a second board provides extra headroom.



Look at the table. One board, one program: two and a half flows per second. One board, two programs: about eighteen. That is seven times faster, and it uses the least energy. Two boards: twenty-two, but twice the energy. So splitting the work matters most. More hardware helps less.

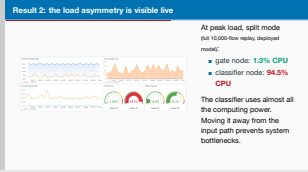
Result 2: the load asymmetry is visible live



At peak load, split mode (full 16,000-flow replay, deployed model):

- gate node: **1.3% CPU**
- classifier node: **94.5% CPU**

The classifier uses almost all the computing power. Moving it away from the input path prevents system bottlenecks.



Here is the live view. Board one uses one percent CPU. Board two uses ninety-five percent. The classifier does almost all the work.

Result 3: what the gate costs, end to end

Mode	Attacks	Missed: gate / clf	FP	F1
A: full replay	1,516	463 / 0	0	**0.820**
A1: split, one-board	992	279 / 0	0	0.813
C: Spark cluster	381	115 / 0	0	0.822
Gate off	242	– / 0	0	1.000

Final verdicts of the deployed pipeline against the replay labels. Full replay: all 10,000 flows once, 20 flows/s for 500 s. Classifier alone, offline: F1 0.995.

- **Precision 100%**: no false alarm on 8,483 benign flows.
- **Recall 69.5%**: every missed attack was *skipped by the gate*. The classifier made no error on what it received.
- The gate's threshold alone sets the end-to-end F1.

The capacity gain is paid for in recall: the gate is a **tunable trade-off** (operating curve in backup), not a free filter.

What do we lose? Precision is one hundred percent. No false alarms. But recall is about seventy percent. The gate drops some attacks. The classifier makes no mistakes. So the gate setting controls this trade-off.

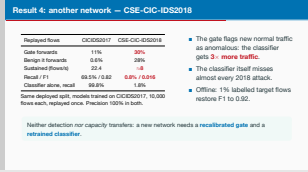
Result 4: another network — CSE-CIC-IDS2018

Replayed flows	CICIDS2017	CSE-CIC-IDS2018
Gate forwards	11%	**30%**
Benign it forwards	0.6%	28%
Sustained (flows/s)	22.4	**≈8**
Recall / F1	69.5% / 0.82	**0.8% / 0.016**
Classifier alone, recall	99.8%	1.8%

Same deployed split, models trained on CICIDS2017, 10,000 flows each, replayed once. Precision 100% in both.

- The gate flags new normal traffic as anomalous: the classifier gets **3× more traffic**.
- The classifier itself misses almost every 2018 attack.
- Offline: 1% labelled target flows restore F1 to 0.92.

Neither detection *nor capacity* transfers: a new network needs a **recalibrated gate** and a **retrained classifier**.



Now we test on the 2018 data, with no retraining. Two problems. One: the gate sends thirty percent of traffic, not eleven. So speed drops from twenty-two to eight. Two: the classifier misses almost all attacks. So for a new network, we must retrain both parts.

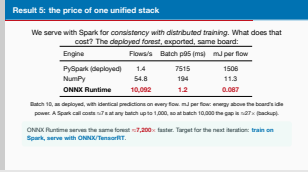
Result 5: the price of one unified stack

We serve with Spark for *consistency with distributed training*. What does that cost? The *deployed forest*, exported, same board:

Engine	Flows/s	Batch p95 (ms)	mJ per flow
PySpark (deployed)	1.4	7515	1506
NumPy	54.8	194	11.3
ONNX Runtime	**10,092**	**1.2**	**0.087**

Batch 10, as deployed, with identical predictions on every flow. mJ per flow: energy above the board's idle power. A Spark call costs ≈7 s at any batch up to 1,000, so at batch 10,000 the gap is ≈27 × (backup).

ONNX Runtime serves the same forest ≈**7,200**× faster. Target for the next iteration: **train on Spark, serve with ONNX/TensorRT**.



We use Spark to match our training. But Spark is slow: about seven seconds for each call. With the same model, ONNX is about seven thousand times faster. So next, we train with Spark, and run with ONNX.

Conclusion

What we showed

- Reproducible edge IDS: Kafka + PySpark on Jetson Orin Nano Super.
- Decoupled stages: **2.5 → 17.8 flows/s** on one board (0.32 J), **22.4** on two (0.60 J), e2e p95 ≈7 s.
- End to end: **precision 100%**, **recall 69.5%**, F1 0.82. Every miss is a gate skip.
- ONNX serves the same forest ≈7,200× faster.

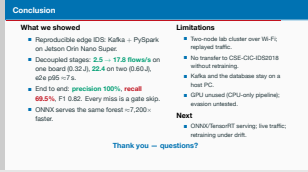
Limitations

- Two-node lab cluster over Wi-Fi; replayed traffic.
- No transfer to CSE-CIC-IDS2018 without retraining.
- Kafka and the database stay on a host PC.
- GPU unused (CPU-only pipeline); evasion untested.

Next

- ONNX/TensorRT serving; live traffic; retraining under drift.

Thank you — questions?


Conclusion
What we showed
Reproducible edge IDS: Kafka + PySpark on Jetson Orin Nano Super.
Decoupled stages: 2.5 → 17.8 flows/s on one board (0.32 J), 22.4 on two (0.60 J), e2e p95 ≈7 s.
End to end: precision 100%, recall 69.5%, F1 0.82. Every miss is a gate skip.
ONNX serves the same forest ≈7,200× faster.
Limitations
Two-node lab cluster over Wi-Fi; replayed traffic.
No transfer to CSE-CIC-IDS2018 without retraining.
Kafka and the database stay on a host PC.
GPU unused (CPU-only pipeline); evasion untested.
Next
ONNX/TensorRT serving; live traffic; retraining under drift.
Thank you — questions?

To sum up. One: splitting the work gives seven to nine times more speed. Two: the gate misses about thirty percent of attacks. Three: a new network needs retraining. Four: ONNX is much faster than Spark. Thank you. Questions are welcome.

Key references

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **edge setting**

[3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **anomaly gate**

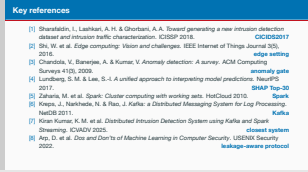
[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP Top-30**

[5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**

[6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**

[7] Kiran Kumar, K. M. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **closest system**

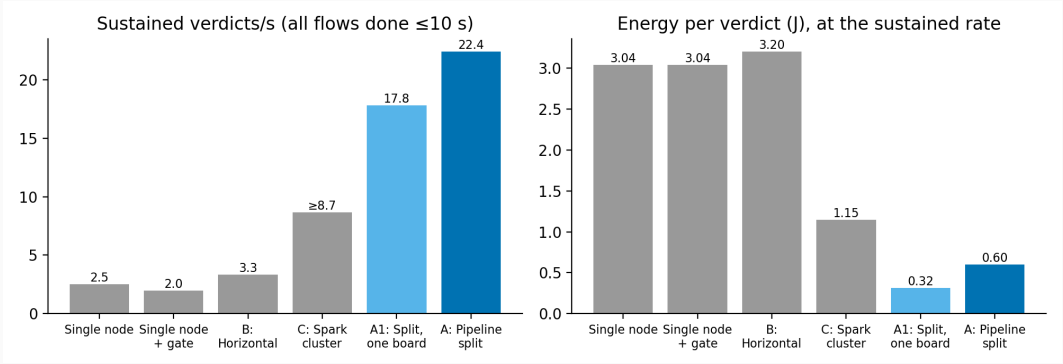
[8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **leakage-aware protocol**



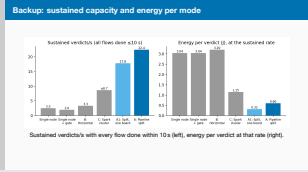
Not presented — shown while taking questions, so sources are on screen if the audience asks. Full list is in the paper.

Backup slides

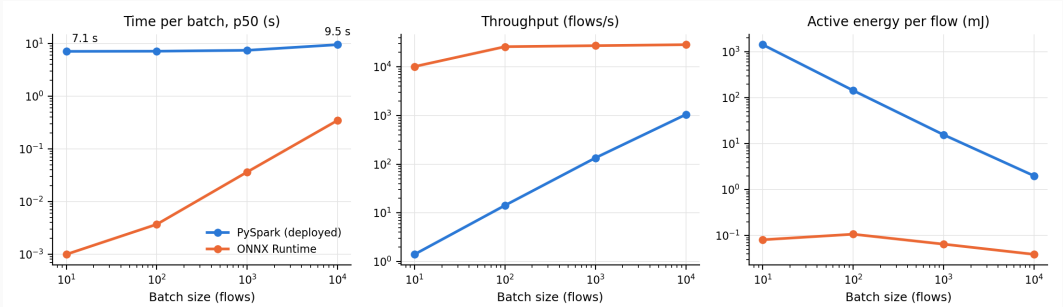
Backup: sustained capacity and energy per mode



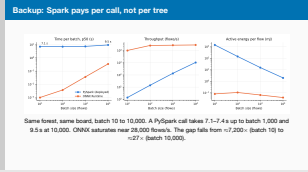
Sustained verdicts/s with every flow done within 10 s (left), energy per verdict at that rate (right).



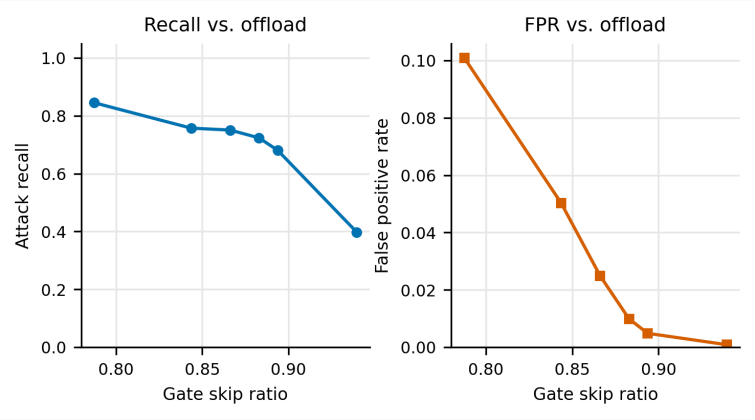
Backup: Spark pays per call, not per tree



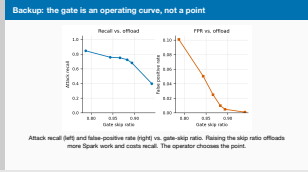
Same forest, same board, batch 10 to 10,000. A PySpark call takes 7.1–7.4 s up to batch 1,000 and 9.5 s at 10,000. ONNX saturates near 28,000 flows/s. The gap falls from ≈7,200× (batch 10) to ≈27× (batch 10,000).



Backup: the gate is an operating curve, not a point



Attack recall (left) and false-positive rate (right) vs. gate-skip ratio. Raising the skip ratio offloads more Spark work and costs recall. The operator chooses the point.



Backup: which model, and why

Feature set	Features	XGBoost F1	RF F1	XGBoost size (MB)	Edge rationale
All features (baseline)	77	**0.9971**	0.9949	2.32	Reference
SHAP Top-30	30	0.9970	**0.9955**	2.37[1]	Explainable, deployed
RF-importance Top-30	30	0.9922	0.9879	2.52	Embedded ranking
PCA k=40	40	0.9939	0.9857	3.82	Unsupervised

We deploy **SHAP Top-30 with Random Forest** (exported model: test F1 0.9959, 30 features): a native Spark ML estimator, so the `PipelineModel` loads on ARM64 without the native libraries Spark's XGBoost integration needs. The SHAP ranking comes from the 77-feature XGBoost model and is computed after `destination_port` is removed.

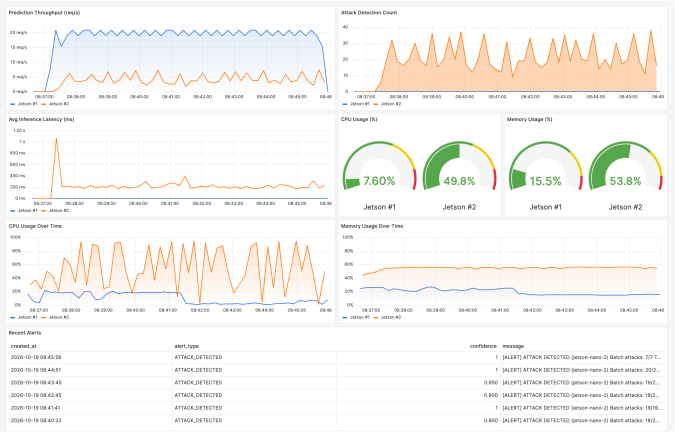
[1]Measured on a single-node retrain (the cluster save kept only metadata).

Backup: which model, and why

Feature set | Features | XGBoost F1 | RF F1 | XGBoost size (MB) | Edge rationale
All features (baseline) | 77 | **0.9971** | 0.9949 | 2.32 | Reference
SHAP Top-30 | 30 | 0.9970 | **0.9955** | 2.37[1] | Explainable, deployed
RF-importance Top-30 | 30 | 0.9922 | 0.9879 | 2.52 | Embedded ranking
PCA k=40 | 40 | 0.9939 | 0.9857 | 3.82 | Unsupervised

We deploy **SHAP Top-30 with Random Forest** (exported model: test F1 0.9959, 30 features): a native Spark ML estimator, so the `PipelineModel` loads on ARM64 without the native libraries Spark's XGBoost integration needs. The SHAP ranking comes from the 77-feature XGBoost model and is computed after `destination_port` is removed.
[1]Measured on a single-node retrain (the cluster save kept only metadata).

Backup: full monitoring dashboard



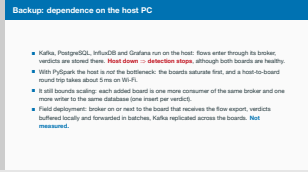
Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards, PostgreSQL-backed alerts.

Backup: full monitoring dashboard

Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards, PostgreSQL-backed alerts.

Backup: dependence on the host PC

- Kafka, PostgreSQL, InfluxDB and Grafana run on the host: flows enter through its broker, verdicts are stored there. **Host down ⇒ detection stops**, although both boards are healthy.
- With PySpark the host is *not* the bottleneck: the boards saturate first, and a host-to-board round trip takes about 5 ms on Wi-Fi.
- It still bounds scaling: each added board is one more consumer of the same broker and one more writer to the same database (one insert per verdict).
- Field deployment: broker on or next to the board that receives the flow export, verdicts buffered locally and forwarded in batches, Kafka replicated across the boards. **Not measured.**



Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects every verdict.
- Each verdict stores the flow's send time and replay row: latency is send to verdict, *queueing included*. Skip ratio and attack recall come from the flows themselves.
- Rate sweep per mode. A rate is sustained when every flow sent in the 45 s window gets its verdict within 10 s after it. 2 runs per rate.
- Fresh Kafka consumer group per run, so no run inherits a backlog.
- Tiers batch 20 flows or flush after 1 s.
- Energy: `tegrastats`, 30 s idle baseline then load at the sustained rate. Two-node modes sum both boards.

